ARENAS

#1 Plantas venenosas

Problema:
En un jardín hay varias plantas. Cada una de ellas ha sido tratada con cierta cantidad de pesticida. Al final de cada día, si alguna planta tiene más pesticida que la que está a su izquierda, y por lo tanto es más débil, muere.

Se te proporcionan los valores iniciales del pesticida en cada una de las plantas. Determina el número de días después de los cuales ninguna planta muere, es decir, el tiempo después del cual ninguna planta tiene un contenido de pesticida mayor que la planta a su izquierda.

Ejemplo:

// niveles de pesticidas

Utilice un-matriz indexada. El día, plantasymorir dejando. En el día, plantaenmuere dejandoNo hay ninguna planta con una concentración de pesticida mayor que la que está a su izquierda, por lo que las plantas dejan de morir después de un día..

Descripción de la función:
Complete la función poisonousPlants en el editor a continuación.

poisonousPlants tiene los siguientes parámetros:

int p[n] : los niveles de pesticidas en cada planta
Devuelve
- int : el número de días hasta que las plantas dejen de morir por pesticidas.

Formato de entrada:
La primera línea contiene un número entero y el tamaño de la matriz
La siguiente línea contiene números enteros separados por espacios.

Entrada de ejemplo
7
6 5 8 4 7 10 9

Salida de muestra
2

Explicación:

Inicialmente, todas las plantas están vivas.

Plantas = {(6,1), (5,2), (8,3), (4,4), (7,5), (10,6), (9,7)}

Plantas[k] = (i,j) => la planta j tiene una cantidad de pesticida = i.

Tras el primer día , quedan 4 plantas, ya que las plantas 3, 5 y 6 mueren.

Plantas = {(6,1), (5,2), (4,4), (9,7)}

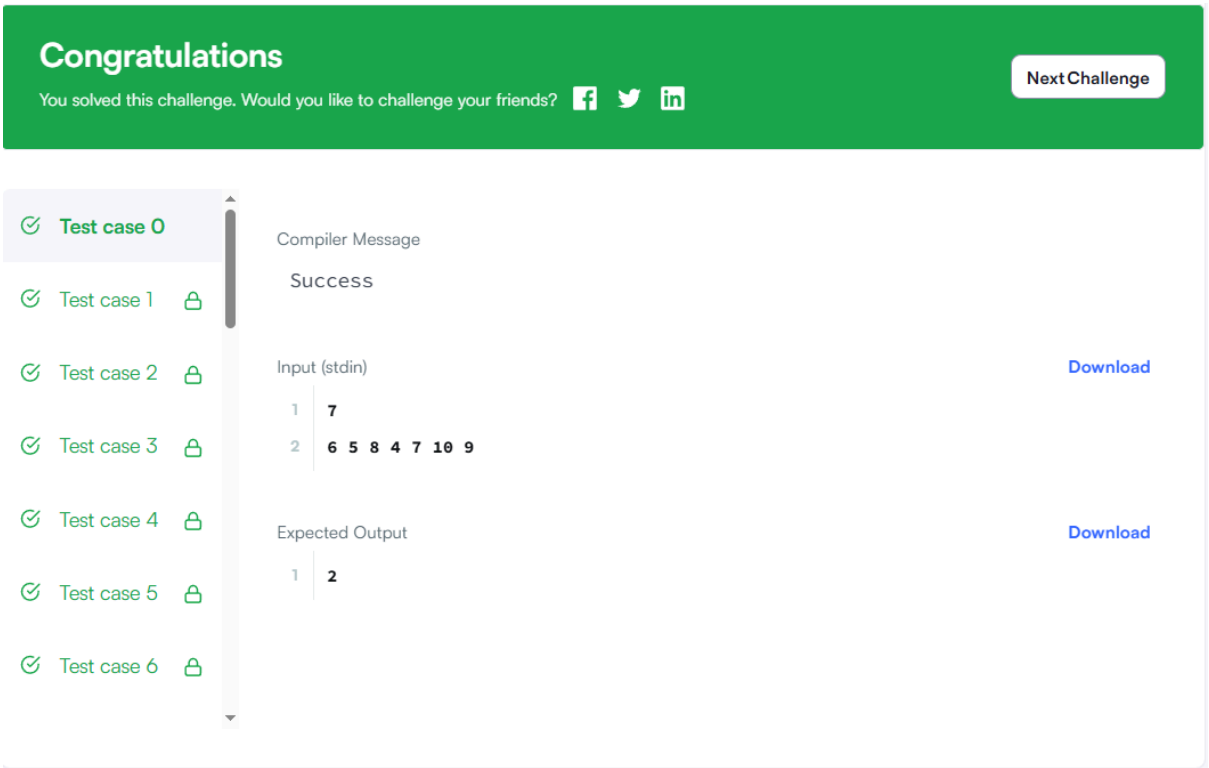
Tras el segundo día, sobreviven 3 plantas, mientras que la planta 7 muere.

Plantas = {(6,1), (5,2), (4,4)}

Las plantas dejan de morir después del segundo día .

Solución Codigo:

```
def poisonousPlants(p):
    stack = []
    max_days = 0

    for plant in p:
        days = 0

        while stack and plant <= stack[-1][0]:
            days = max(days, stack[-1][1])
            stack.pop()

        if not stack:
            days = 0
        else:
            days += 1

        max_days = max(max_days, days)
        stack.append((plant, days))

    return max_days


n = int(input())
p = list(map(int, input().split()))

result = poisonousPlants(p)
print(result)

}
```



#2 Cola usando dos pilas

Problema:
Una cola es un tipo de dato abstracto que mantiene el orden en que se añadieron los elementos, lo que permite eliminar los elementos más antiguos al principio y añadir los nuevos al final. Esto se denomina estructura de datos FIFO (primero en entrar, primero en salir) porque el primer elemento añadido a la cola (es decir, el que lleva más tiempo esperando) siempre es el primero en eliminarse.

Una cola básica tiene las siguientes operaciones:
- Poner en cola : agrega un nuevo elemento al final de la cola.
- Dequeue : elimina el elemento del frente de la cola y lo devuelve.

Formato de entrada:
La primera línea contiene un solo entero,, que indica el número de consultas.
Cada línea i de q. Las líneas subsiguientes contienen una sola consulta en el formato descrito en el enunciado del problema anterior. Las tres consultas comienzan con un entero que denota la consulta. type, pero solo consulta 1 va seguido de un valor adicional separado por espacios, x, que indica el valor que se pondrá en cola.

Restricciones:
- $1 \leq q \leq 10^5$
- $1 \leq type \leq 3$
- $1 \leq |x| \leq 10^9$
- Se garantiza que siempre exista una respuesta válida para cada consulta de tipo 3.

Entrada de ejemplo:
Función STDIN
----- --------
10 q = 10 (número de consultas)

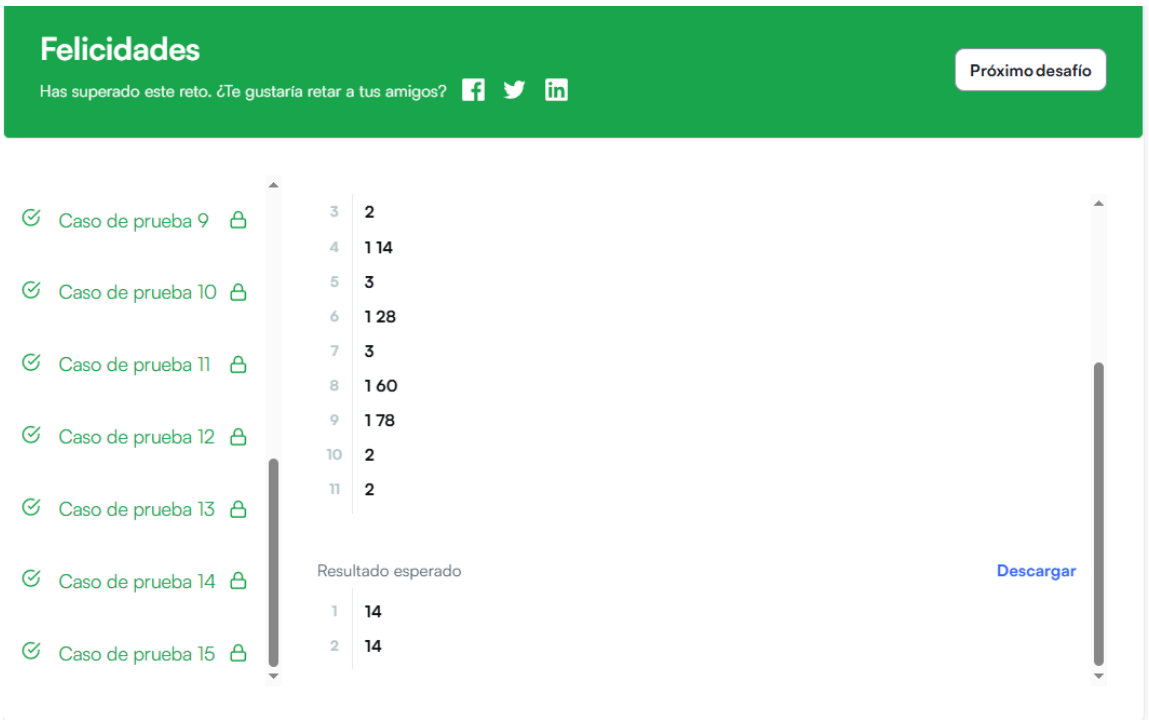
1 42 1.ª consulta, poner en cola 42
2 elementos frontales de desencolar
1 14 poner en cola 42
3. Imprime el elemento frontal
1 28 poner en cola 28
3. Imprime el elemento frontal
1 60 poner en cola 60
1 78 poner en cola 78
2 elementos frontales de desencolar
2 elementos frontales de desencolar

Salida de muestra:

14
14

Solución Codigo:

```
stack1 = []
stack2 = []
q = int(input())

for _ in range(q):
    query = list(map(int, input().split()))

    if query[0] == 1:
        stack1.append(query[1])

    elif query[0] == 2:
        if not stack2:
            while stack1:
                stack2.append(stack1.pop())
        stack2.pop()

    elif query[0] == 3:
        if not stack2:
            while stack1:
                stack2.append(stack1.pop())
        print(stack2[-1])
```



#3 And xor OR

Problema:
Dado un arreglo A [] de N elementos distintos. Sean M_1 y M_2 el elemento más pequeño y el siguiente más pequeño en el intervalo [L, R], donde 1 ≤ L < R ≤ N.

$$S_i = (((M_1 \wedge M_2) \oplus (M_1 \vee M_2)) \wedge (M_1 \oplus M_2))$$

Donde ∧, ∨, ⊕ Son operadores bit a bit AND, OR y XOR respectivamente su tarea es encontrar el valor máximo posible de S_i.

Formato de entrada:
La primera línea contiene un número entero N. La segunda línea contiene N enteros, que representan elementos de la matriz A[].

Restricciones:

- $1 < q \leq 10^6$
- $1 \leq A_i \leq 10^9$

Formato de salida:
Imprime el valor del máximo valor posible de S_i

Entrada de ejemplo:

5
9 6 3 5 2

Solución Código:

```python
#!/bin/python3
import math
import os
import random
import re
import sys

def andXorOr(a):
    stack = []
    max_val = 0

    for num in a:
        while stack:
            max_val = max(max_val, num ^ stack[-1])

            if stack[-1] > num:
                stack.pop()
            else:
                break
        stack.append(num)
    return max_val

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    a_count = int(input().strip())

    a = list(map(int, input().rstrip().split()))

    result = andXorOr(a)

    fptr.write(str(result) + '\n')

    fptr.close()
```

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

- Test case 23
- Test case 24
- Test case 25
- **Test case 26**
- Test case 27
- Test case 28
- Test case 29
- Test case 30

Compiler Message

```
Success
```

Input (stdin) Download

```
5
9 6 3 5 2
```

Expected Output Download

```
15
```